

Simulation Demo

In this vignette, we provide a worked simulation example for the **BSGL** package using the prepared simulation data set with $n = 1000$ and $m = 5$. The example demonstrates how to fit the proposed BSGL model and compare prediction with GSVC, GGP-GAM, and MGWR.

The same comparison is implemented in `run_demo.R`, which installs and loads the local package, refits the models locally, and writes the result table. This demo is a single local run. The manuscript-reference simulation values are the retained 10-replicate summaries in `results/simulation/`: `rep10_metrics_by_replicate.csv`, `rep10_scp_by_replicate.csv`, `rep10_metrics_summary.csv`, and `rep10_scp_summary.csv`. Because the demo scripts refit the models locally, small numerical differences across local runs may occur due to runtime settings, stochastic fitting components, or seed/split differences.

Required Packages

The code below loads the package and the comparison-method dependencies used in the paper example.

```
## =====  
## load packages  
## =====  
library(BSGL)  
library(mgcv)  
library(GWmodel)  
library(sp)
```

Simulated Data

The prepared simulation data are saved in `data/simulation_demo/n1000_p5`. The training data contain $n = 1000$ observations and $m = 5$ covariates; the test data contain 200 held-out observations.

```
## =====  
## load the data  
## =====  
load("data/simulation_demo/n1000_p5/train_data.RData")  
load("data/simulation_demo/n1000_p5/test_data.RData")  
load("data/simulation_demo/n1000_p5/grid_data.RData")  
load("data/simulation_demo/n1000_p5/meta_info.RData")  
  
dim(train_data$X)  
dim(test_data$X)  
meta_info
```

BSGL Method

The proposed method represents each spatially varying coefficient surface with a tensor-product B-spline basis and applies Bayesian group-lasso shrinkage to the basis coefficients. The example below uses the same simulation-demo settings as `run_demo.R`.

```
## =====  
## fit BSGL
```

```
## =====
set.seed(20260601)

bsgl_fit <- fit_bsgl_multichain(
  y = train_data$y,
  X = train_data$X,
  coords = train_data$coords,
  L = 25,
  niter = 1000,
  nburn = 100,
  a_lam = 15,
  b_lam = 1,
  nchains = 2,
  parallel = FALSE,
  verbose = TRUE
)

bsgl_pred <- pred_bsgl(bsgl_fit, test_data$X, test_data$coords)
bsgl_beta <- betas_bsgl(bsgl_fit, list(grid_points = train_data$coords))
```

Comparison Methods

The same data set can be fit with the non-selection spatially varying coefficient model GSVC, the GGP-GAM comparison, and MGWR.

```
## =====
## fit comparison methods
## =====

gs_fit <- fit_gs(
  y = train_data$y,
  X = train_data$X,
  coords = train_data$coords,
  L = 25,
  niter = 1000,
  nburn = 100,
  verbose = FALSE
)

gs_pred <- pred_gs(gs_fit, test_data$X, test_data$coords)
gs_beta <- betas_gs(gs_fit, list(grid_points = train_data$coords))

gam_model <- fit_gam(train_data$X, train_data$y, train_data$coords)
gam_pred <- pred_gam(gam_model, test_data$X, test_data$coords)
gam_beta <- get_gam_betas(gam_model, train_data$coords)

mgwr_model <- fit_mgwr(train_data$X, train_data$y, train_data$coords)
mgwr_pred <- pred_mgwr(mgwr_model, test_data$X, test_data$coords)
mgwr_beta <- get_mgwr_betas(mgwr_model, train_data$coords)
```

Method Comparison

The following code computes test-set prediction error and coefficient-surface error for the simulation example. The null-surface error MSE_0 is useful for checking whether a method overfits inactive covariates.

```

## =====
## prediction and coefficient-surface comparison
## =====
true_beta <- calc_true_beta(train_data$coords, ncol(train_data$X))

metrics <- data.frame(
  Method = c("BSGL", "GSVC", "GGP-GAM", "MGWR"),
  MSPE = c(
    mean((test_data$y - bsgl_pred$mean)^2),
    mean((test_data$y - gs_pred$mean)^2),
    mean((test_data$y - gam_pred)^2),
    mean((test_data$y - mgwr_pred)^2)
  ),
  Coverage = c(
    mean(test_data$y >= bsgl_pred$lower & test_data$y <= bsgl_pred$upper),
    mean(test_data$y >= gs_pred$lower & test_data$y <= gs_pred$upper),
    NA,
    NA
  ),
  MSE_1 = c(
    mean((true_beta[, 1:3] - bsgl_beta[, 1:3])^2),
    mean((true_beta[, 1:3] - gs_beta[, 1:3])^2),
    mean((true_beta[, 1:3] - gam_beta[, 1:3])^2),
    mean((true_beta[, 1:3] - mgwr_beta[, 1:3])^2)
  ),
  MSE_0 = c(
    mean((true_beta[, 4:5] - bsgl_beta[, 4:5])^2),
    mean((true_beta[, 4:5] - gs_beta[, 4:5])^2),
    mean((true_beta[, 4:5] - gam_beta[, 4:5])^2),
    mean((true_beta[, 4:5] - mgwr_beta[, 4:5])^2)
  )
)

metrics

```

Credible-Effect Maps and SCP

BSGL and GSVC provide posterior samples for coefficient surfaces. Spatial coverage probability (SCP) is computed as the proportion of grid locations where the pointwise credible interval excludes zero.

```

## =====
## compute SCP
## =====
bsgl_scp <- calc_scp_bsgl(bsgl_fit, grid_data, ci_level = 0.95)
gs_scp <- calc_scp_gs(gs_fit, grid_data, ci_level = 0.95)

scp <- data.frame(
  variable = paste0("beta", seq_len(ncol(train_data$X))),
  BSGL = as.numeric(bsgl_scp),
  GSVC = as.numeric(gs_scp)
)

scp

```

For the package-based local demo result table, run:

```
## =====  
## package-based result script  
## =====  
system("Rscript run_demo.R")  
read.csv("results/demo_method_comparison.csv")
```

The retained CSV files under **results/** provide the manuscript-reference values reported in the paper.